Capital Circuit - Monopoly Game

CSC301 Group 4 | console Monopoly in Java (Abu Dhabi edition)

AI Usage Log for user CS (Caron Sylvia)

Course	CSC301 Data Structures and Algorithms - Dr Zakaria Khiati, Abu Dhabi University
Group / member	Group 4 | CS (Caron Sylvia)
AI tool	Claude (model claude-opus-5-5) in the Claude Project "Monopoly-Game" (conversations 01 and 03); Prompts 01.19-01.21 and 01.26-01.27 in a separate Claude desktop session (conversation 02)
Period	1 November 2026, 16:09-19:59 GST (28 prompts) and 6 November 2026, 14:00-19:00 GST (4 prompts) | 32 prompts, 3 conversations
Source log	C:\Claude\Monopoly\AI-Usage-log.txt (prompts verbatim, AI actions, AI responses, reflection candidates)
Brief reference	Section 7 (AI use and the AI log, 20% of the grade); Sections 7.1-7.3

1. Summary

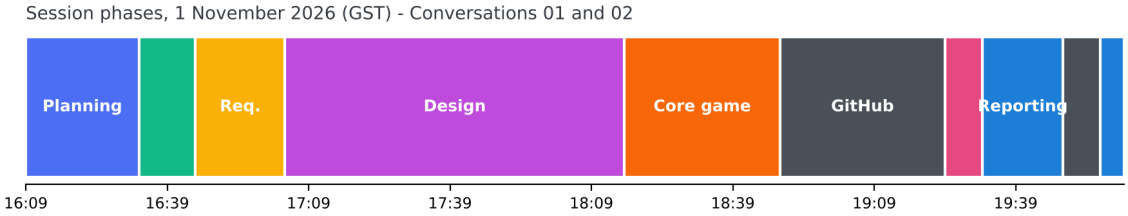
1.1 How AI was used

CS used Claude as the single AI tool of record for the group's Monopoly assignment, inside the Claude Project that holds the brief. Every instruction was logged verbatim in **AI-Usage-log.txt** together with what the AI did and how it answered, as Section 7 of the brief requires. The work moved in ten phases: planning, environment set-up, clarifying requirements, design, the core game, GitHub, enhancements, reporting and, on 6 November, the twists and the move to an Eclipse-only build.

The AI was given the brief's exact rules, constraints and method signatures rather than a general request, and work proceeded in small steps: no code was written until CS gave the go-ahead (Prompt 01.13). The AI produced the design (UML class diagram, board and card files), the Java source, the console board view and colour scheme, the build and test scripts, the GitHub sync script and, in conversation 03, the three twists and the Eclipse project. CS made the key decisions: deferring the twists and later choosing T1 Reverse, T3 Expansion and T7 Recall, the bonus features, the visual console and player colours after play-testing, keeping a real e-mail out of the public repository, and building and running only in Eclipse.

Every AI output was checked rather than trusted: 96 automated tests (including each Section 8.2 scenario and the T1 scenario, reproduced with scripted dice), 1,000 random seeded games, and an automatic constraint check for allowed imports, banned collections and loop-free recursive methods, extended to the twist methods and ported to Java so it runs in Eclipse. This surfaced nine cases where the AI was wrong or incomplete (Section 2.3), each detected and fixed and recorded for the group's reflection.

32	3	27	96	1,000	9
prompts logged	conversations	Java source files	automated tests passing	seeded games checked	AI errors caught



Colours match the phase labels in the timeline (1.2). Conversation 03 (6 November: Twists, Eclipse) is shown in the timeline.

1.2 Timeline of prompts and functionality built

Time (GST)	Prompt	Functionality built / outcome	Phase
16:09	**01.01**	AI log file created; brief analysed	Planning
16:12	**01.02**	Console UI prototype image	
16:23	**01.03**	Java toolchain check - JDK missing	
16:27	**01.04**	Build approach: Claude Project + VS Code	
16:33	**01.05**	One-click JDK 25 installer + setup skill	Environment
16:43	**01.06**	Environment verified; VS Code pinned to JDK 25	
16:45	**01.07**	Twist pool (T1-T8) explained	Requirements
16:47	**01.08**	Simplest twists shortlisted	
16:53	**01.09**	Twists deferred; bonus rules reviewed	
16:58	**01.10**	Language-toggle bonus assessed	
17:02	**01.11**	Bonus chosen: B4 visualiser + language	
17:04	**01.12**	UML class diagram, board and card files	Design
18:16	**01.13**	Core game: 24 classes, 44 tests, checker	Core game
18:35	**01.14**	Console board view (prototype look)	
18:45	**01.15**	Colour mode: player colours on squares	
18:49	**01.16**	GitHub sync script	GitHub
18:54	**01.17**	Push error 403 diagnosed; account switch	
18:57	**01.18**	Private no-reply e-mail for commits	
~19:02	**01.19**	Repo link checked (separate session)	
~19:07	**01.20**	Signed in with team account (separate session)	
19:12	**01.21**	Project pushed to CS-Monopoly branch	
19:24	**01.22**	Standings with properties; quit screen	Enhancements
19:32	**01.23**	AI usage PDF report	Reporting
19:45	**01.24**	Data structures report (abandoned, re-sent)	
19:47	**01.25**	Data structures PDF report	
~19:50	**01.26**	.vscode settings kept local (separate session)	GitHub
~19:53	**01.27**	Changes checked and pushed (separate session)	
19:59	**01.28**	Prompt references corrected; log completed	Reporting
		6 November 2026	
14:00	**03.01**	Twists ranked; T1 + T3 + T7 recommended	Twists
14:02	**03.02**	T1, T3, T7 built; 93 tests; docs and log	
18:54	**03.03**	Eclipse set-up steps on another laptop	Eclipse
19:00	**03.04**	Eclipse-only: project, Run entries, .bat removed	

~ marks an approximate time: Prompts 01.19, 01.20, 01.26 and 01.27 were logged without a timestamp by the separate Claude session (conversation 02).

2. Detailed report

2.1 Log index (brief Section 7.1)

No.	Date	Member	Tool	What the conversation was for	Link / file name
01	01-11-2026	CS (Caron Sylvia)	Claude (claude-opus-5-5), Claude Project "Monopoly-Game"	Prompts 01.01-01.18, 01.22-01.25 and 01.28: brief analysis, AI log, prototype, Java toolchain, environment set-up, twists and bonus, design, core game, board view, colour mode, GitHub sync script, standings, both PDF reports	session_01Rc15ip9Mj11V2i3BSohRjQ; shared link / PDF export: **to be added**
02	01-11-2026	CS (Caron Sylvia)	Claude, separate desktop-app session	Prompts 01.19-01.21 and 01.26-01.27: check the GitHub link, sign in with the team account, push the project to branch CS-Monopoly, keep .vscode settings local, check and upload changes	Shared link / PDF export: **to be added**
03	06-11-2026	CS (Caron Sylvia)	Claude (claude-opus-5-5), Claude Project "Monopoly-Game"	Prompts 03.01-03.04: choose the twists; build T1 Reverse, T3 Expansion, T7 Recall; tests; design notes, UML, README, both PDF reports and the log; Eclipse-only build and run	session_01Rc15ir1HbnsyYqbzoLnA8H; shared link / PDF export: **to be added**

The text log numbers the prompts of conversations 01 and 02 together as 01.xx, in the order they were logged; conversation 02 is the separate desktop session. Row 02 was added to the text log's Part A on 6 November so that the text log and this report agree. Section 7.1 asks for one unedited export per conversation: all three export links must be added before submission.

2.2 Prompt-by-prompt log

Prompts are quoted verbatim from AI-Usage-log.txt (spelling unchanged). The outcome column summarises the AI actions and response recorded there; the full text stays in the log file.

No.	Time (GST)	Phase	Prompt by CS (verbatim)	AI output and outcome
01.01	16:09	Planning	How this monopoly game must be developed is uploaded to the context section as a word document. Refer to the instructions and start creating the AI Log file, including this prompt. let me know the file path of the AI log file. Dont start developing the code, until i tell so. just read the context and keep in youe memory.	Read the brief from the Project; created AI-Usage-log.txt with the three parts the brief requires (index, full log, reflection). No code, as instructed.
01.02	16:12	Planning	now show me a prototype image of how this monopoly game will look like. generate this image based on the context file and referring to good monopoly games online. log this also	Looked at open-source terminal Monopoly projects; designed an original 24-square Abu Dhabi board; rendered a console mock-up (Prototype_Console_Mockup.png). Flagged that its money figures were illustrative.
01.03	16:23	Planning	Now show me the software components of JAVA that is used to build this monopoly game? Check my computer and let me know if i have the required software to build and run this game.	Listed the Java components the game needs; scanned folder names on the PC and reported no JDK; recommended Temurin JDK 25 LTS. Missed a JDK 17 on drive F: (reflection case 1).
01.04	16:27	Planning	ok then advise me how to build this game project? can i continue in Claude project or should i switch to VScode with Java extensions? or should i follow a hybrid approach, where claude project will suggest me the required AI skills which i can use in VS code?	Advised a hybrid: this Claude Project as the only AI of record, VS Code for build and debug with its own AI switched off (brief 7.3); proposed a phased build plan.
01.05	16:33	Environment	ok i will continue to use this Claude Project only for this monopoly development. Pls install the required software on this computer. Call this section as software environment setup and this entire activity create as a skill file (give appropriate naming for this skill file) in the project folder.	Created a one-click installer (Setup-Environment.bat/.ps1), AI-off VS Code settings and the csc301-java-environment-setup skill. Could not run the installer itself (no terminal input, UAC).
01.06	16:43	Environment	i had run the setup\Setup-Environment.bat look like required java components are installed and paths are setup. Pls verify and update me. check the vs code part also...i have created a subfolder under monopoly folder called code for the source code for this project.	Checked the installer report: Temurin 25.0.5 LTS, javac, JAVA_HOME, VS Code Java extensions, Git, compile test all passed. Found the extra JDK 17; pinned VS Code to JDK 25; added Check-Java.bat.
01.07	16:45	Requirements	advise me on the group's three twists, i didnt get it...what i need to do for this?	Explained the pool of eight twists and that the instructor assigns three per group on Blackboard; asked CS to confirm the group number and twists.

No.	Time (GST)	Phase	Prompt by CS (verbatim)	AI output and outcome
01.08	16:47	**Requirements**	i want to use only simple ones for the twists without disrupting too much and easy to implement. so suggest me 3 twists	Challenged choosing twists (they are assigned); if a choice is allowed, recommended T1 Reverse, T3 Expansion, T7 Recall.
01.09	16:53	**Requirements**	Ok for now lets not build the twists, will enquire about this what twist is required for this group and we will enhance this later. Let me know about the bonus mark and what are we planning to do for that?	Recorded the decision to defer twists; explained the bonus rules (0.25 each, unique, judged in demo order) and proposed five candidate features.
01.10	16:58	**Requirements**	What about adding a feature to change the monopoly game language as options for this bonus. there will be toggle to select language like english, french, arabic. whats your view on this? since the mark is only .25 should be touch complex parts	Assessed the language toggle: little data-structure content, ResourceBundle/HashMap banned, Arabic console risk; suggested an own recursive binary-search lookup.
01.11	17:02	**Requirements**	Ok lets use option B4 and also language toggle option for bonus. keep this in memory	Recorded the bonus choice: B4 pointer visualiser and a language toggle (English, French, Arabic); saved the decisions to project memory.
01.12	17:04	**Design**	ok now lets move to the next step the core design and build the uml class diagram, plus board and card files	Class model to the Section 5 signatures; UML diagram; own board.txt (24 squares) and cards.txt (20 cards) checked by script; design notes D1-D13.
01.13	18:16	**Core game**	ok now, based on this design build the monopoly game application using java and requirements mentioned in the context document	Built the core game (24 Java files) with the 8 required recursive methods and no loops; 44 tests incl. every Section 8.2 scenario; constraint checker; build/run/test scripts; README. Found and fixed 3 AI issues.
01.14	18:35	**Core game**	i dont understand why it is a command like game, i thought there will be visualisation and it will be a console game like the gone shown in the image initially.	Acknowledged the miss (the prototype showed a board); added BoardView: ring of squares, owners, tokens, player table, turn queue, events in plain words. --text keeps the Appendix C log mode. 50 tests.
01.15	18:45	**Core game**	the console game is running in black and white mode, like shown in the prototype image, can you pls make the game in color mode, so that it looks atheistically better. choose different color for different players and the use that color for the property they own. make these enhancements	Switched on colour in the classic Windows console; each player has a colour that tints the squares they own; changed the palette after a green-on-green clash.
01.16	18:49	**GitHub**	now i need this project to be synced with the github branch created for the person in the project group. Pls sync the changes to https://github.com/cb-beep-netizen/CSC301-Group4-Monopoly/tree/CS-Monopoly	GitHub not linked to the session; read the public repository; found teammates' files and twists T1/T5/T8 in a teammate's log; wrote sync-to-github.bat (never deletes teammates' files, keeps private files out).
01.17	18:54	**GitHub**	check the error screenshot, the github signin page did not open...pls fix this (Attachment: screenshot of sync-to-github.bat ending "SYNC FAILED - see sync-log.txt" after step [5/5] Pushing to GitHub.)	Diagnosed HTTP 403: the saved GitHub account had no write access. The script now explains the fixes and can switch account.
01.18	18:57	**GitHub**	yes, i want to keep my email private, pls make the necessary changes so that my email is not exposed	Sync script signs commits with the GitHub no-reply address and re-signs unpushed commits; rehearsed on a copy of the branch.
01.19	not recorded	**GitHub**	Pls connect to the github repo branch https://github.com/cb-beep-netizen/CSC301-Group4-Monopoly/tree/CS-Monopoly and sync the folder C:\claude\monopoly to github branch mentioned in the link	(Separate Claude session, conversation 02) Checked the link; dry-run push failed with 403; flagged the e-mail in the unpushed commit and a changed VS Code AI setting.
01.20	not recorded	**GitHub**	ok login to a different github account which has access. i have already logged in this account using my chrome browser	(Separate Claude session, conversation 02) Signed in with the team account; dry-run push succeeded; re-signing was blocked by a permission check.
01.21	19:12	**GitHub**	yes log this request in AI-usage-log.txt file. try to connec to repo using [e-mail removed from this log: the cb-beep-netizen account] account and copy the monopoly project to the branch https://github.com/cb-beep-netizen/CSC301-Group4-Monopoly/tree/CS-Monopoly	(Separate Claude session, conversation 02) Logged Prompts 01.19-01.21 (e-mail removed); pushed the project to CS-Monopoly with a no-reply address (commits aabd08f, b2578b0).

No.	Time (GST)	Phase	Prompt by CS (verbatim)	AI output and outcome
01.22	19:24	**Enhancements**	I need some enhancements to the program. at the end of the game, Pls show the ranking of the players, how much is their networth and list the properties owned by them. this should come when the game is quit. also in the console centre also currently where the ranking, networth is show, pls add the properties owned by them also.	Centre panel lists each player's properties; quit and game-over screens show the ranking, net worth (cash + property) and full property lists. Q writes no END line (Appendix C). 53 tests.
01.23	19:32	**Reporting**	make a pdf report based on the AI-Usage-log log file. the report must be titled appropriately using the context for the project name, sub heading as AI usage log for user CS (Caron Sylvia). The report should initially show a summary section with a small write up on AI usage and a pictorial timeline of AI prompts with functionalities build. Then the detailed report section should show the tabular view of the AI log usage. as required in the context word doc uploaded initially. keep this pdf report also in the monopoly folder	Generated the first version of this report from AI-Usage-log.txt.
01.24	19:45	**Reporting**	Now make a detailed PDF report on the JAVA data structures used in this monopoly game and explain for which game function it was used. create the report in the same format as AI log report with similar headings, but sub heading should be "Data structures used in the Monopoly Game and its explanation". in the report content clearly explain Game functionality that was built and which Data structure algorithm was used e.g linkedlist or double linkedlist etc...refer to the context doc uploaded initially and prepare this report based on the code and game functionality and data structures used	None: CS interrupted the request and re-sent it with a screenshot (01.25). Logged as an abandoned attempt (brief 7.1).
01.25	19:47	**Reporting**	(Attachment: screenshot of the game in the Windows Command Prompt, turn 18, GAME STOPPED standings for players Caron, Helen and Jay.) Now make a detailed PDF report on the JAVA data structures used in this monopoly game and explain for which game function it was used. create the report in the same format as AI log report with similar headings, but sub heading should be "Data structures used in the Monopoly Game and its explanation". in the report content clearly explain Game functionality that was built and which Data structure algorithm was used e.g linkedlist or double linkedlist etc...refer to the context doc uploaded initially and prepare this report based on the code and game functionality and data structures used. include the attached screenshot from the actual game after the summary portion of the report. so that it gives a feel of how the game looks like. post this image, you include the explanation part.	Confirmed the PC code matched the AI's copy (MD5); wrote the data structures PDF report with CS's screenshot, the structure explanations, recursion table, constraint compliance and data flow.
01.26	not recorded	**GitHub**	pls ignore uploading .vscode/settings.json, i was trying to use claude cowork project module to do it	(Separate Claude session, conversation 02) Marked .vscode/settings.json skip-worktree so syncs no longer upload the local change; noted Cowork use may need logging.
01.27	not recorded	**GitHub**	Pls check from changes to the folder and upload to the github repo branch	(Separate Claude session, conversation 02) Found 7 changed and 3 new files; tests 53/53 and constraint check PASS; scanned for e-mails and PC names; pushed commit 857f200. Flagged a prompt-number mismatch.
01.28	19:59	**Reporting**	yes	Corrected the references from Prompt 01.19 to 01.22 in three source files and note D21 (comments only); logged Prompts 01.26-01.28.
03.01	14:00	**Twists**	Lets now finalize on the 3 twists for the monopoly game. Pls suggest some simple, easy to implement twists	Flagged first that the twists are listed per group on Blackboard; ranked the eight twists by effort; recommended T1 + T3 + T7; listed the edge cases to settle (ORDER line, inserting after the last square, deck size after RECALL).

No.	Time (GST)	Phase	Prompt by CS (verbatim)	AI output and outcome
03.02	14:02	**Twists**	(Attachment: excerpt from the previous AI answer: "T1 + T3 + T7") ok lets do T1, T3 and T7. build this changes to the code and also update the required documentations, audit logs and audit related documentation. in the documentation, have a section for twists and explain the same.	Built T1 (QueueTwists.reverse), T3 (EXPAND lines, insertAfter at turn 20) and T7 (QueueTwists.rotate); 3 EXPAND lines and 4 twist cards; 93 tests and 1,000 seeded games pass; checker extended; README, design notes D22-D27, UML v4, both PDF reports and this log updated.
03.03	18:54	**Eclipse**	how do i make this project run from eclipse directly in a different laptop. the fully monopoly folder is copied to that laptop	Explained how to open the copied folder as an Eclipse project (JDK 21+, no module-info, source folders code and tests, compliance 21, UTF-8) and the run configurations; flagged the Eclipse console limits.
03.04	19:00	**Eclipse**	Can you make it run solely on eclipse? [AI clarifying questions 03.04 - answered by CS] - Scope? -> "Eclipse only, drop .bat" (Eclipse project files, Run menu entries, Eclipse-friendly game, and remove build.bat, run.bat and run-tests.bat). - Push the Eclipse project files to the group repo? -> "Push to the repo". - Default play screen in Eclipse? -> "Board view, redrawn below".	Asked three questions first (scope, GitHub, screen). Built the Eclipse project files and four Run menu entries; no-argument start and --no-clear; dark colour panel for the white console; Java ConstraintCheck; removed the three .bat files with git rm; 96 tests pass.

Prompts 01.01-01.28 are dated 1 November 2026; 03.01-03.04 are dated 6 November 2026.

2.3 AI reflection - candidate cases (brief Section 7.1)

Cases where the AI's output was wrong, incomplete or broke a constraint, recorded as they happened. The brief asks for at least three, with how each was detected and fixed.

No.	Prompt	What the AI produced	How it was detected	How it was fixed
1	01.03, found at 01.06	Stated that no JDK was installed, after checking only standard folders on drive C:.	The installer's "where java" output also listed F:\Android\jdk-17.	VS Code pinned to Temurin 25; Check-Java.bat added; skill now checks other drives.
2	01.02	A prototype screen whose cash and net-worth figures were invented.	Reasoning (flagged by the AI itself).	Marked as illustrative; real values come from seeded test games.
3	01.13	A board whose rents never bankrupt anyone (0 bankruptcies in 300 games of up to 3,000 turns).	Simulation of seeded games.	Rents x2/x3/x4 tested (still 0 in 60 turns); brief-like rents kept; bankruptcy shown with scripted dice; documented.
4	01.13	Test code using java.util.Arrays (not allowed) and a constraint checker with false positives.	Reviewing imports; reasoning about the checker output.	Own list() helper; word-boundary patterns in the checker.
5	01.13, found at 01.14	A game that printed only log lines, although the AI's own prototype showed a console board.	CS played the game and compared it with the prototype (wrong result while playing).	BoardView added as the default screen; --text keeps the Appendix C log mode.
6	03.02	Board-view narration for the T1 ORDER line written with java.util.Arrays.asList - again a class not on the allowed list (compare case 4).	Reasoning: the AI re-read its edit against brief 4.2 before compiling.	StringBuilder loop instead; checker run on a copy with Arrays imported to prove it reports it.
7	03.02	Two wrong twist tests: a PAY_EACH-after-REVERSE test starting on the wrong square, and a "nobody moves" T3 test that recorded positions before turn 20's own dice move.	First test run: 91 passed, 2 failed; then tracing the printed log lines.	Both test set-ups corrected; no game code changed; 93 passed, 0 failed.
8	03.02	Labelled the new session "Conversation 02", although this report already used 02 for the separate desktop session.	Reasoning: reading this report's log index before regenerating it.	Renamed every reference to Conversation 03; row 02 added to the text log's index.
9	03.03, found at 03.04	Said colours work in the Eclipse console, but the board's light text colours were chosen for a black console and are hard to read on Eclipse's default white console.	Reasoning about the palette, then rendering the real colour output on a white background (a trace).	Colour board drawn on its own dark panel with a light default text colour; test added.

Still to be written by the group: the 1-2 page reflection in the group's own words, and the closing paragraph on which parts the group designed or rewrote itself, and why (Section 7.1). Cases 4 and 6 are a good pair for the demo question "which constraint did the AI ignore most often".

2.4 Files produced, traced to prompts

Brief Section 7.3: code that appears in no log, and that no member can explain, is treated as unlogged AI use. Each AI-produced file also names its prompt in its header.

File	Prompt(s)	Purpose
AI-Usage-log.txt	01.01 (every prompt)	Audit trail required by brief Section 7
Prototype_Console_Mockup.png	01.02	Console UI prototype
setup\Setup-Environment.bat/.ps1, Check-Java.bat	01.05, 01.06	JDK 25 installer and checks
skills\csc301-java-environment-setup\SKILL.md	01.05	Reusable set-up procedure
.vscode\settings.json	01.05, 01.06, 01.13, 01.26	Workspace settings (Java paths; AI features)
design\UML_Class_Diagram.*, Core-Design-Notes.txt	01.12-01.22, 03.02	Class diagram v4, design decisions D1-D27
code\data\board.txt, cards.txt	01.12, 03.02	Own board (24 squares + 3 EXPAND lines) and 24 cards incl. REVERSE and RECALL
code*.java (27 files)	01.13-01.15, 01.22, 03.02	Game source incl. BoardView; QueueTwists and Expansion for the twists
tests\GameTests.java, ConstraintCheck.java, check_constraints.py	01.13-01.22, 03.02, 03.04	96 tests; constraint check incl. reverse and rotate (Java for Eclipse; Python optional)
.project, .classpath, .settings\, eclipse*.launch	03.04	Eclipse project and the four Run menu entries (Play, Play with seed, Twist demo, Tests)
README.txt	01.13-01.22, 03.02, 03.04	README deliverable: Eclipse import and run, twists section
build.bat, run.bat, run-tests.bat	01.13 (removed at 03.04)	Former build, run and test scripts; removed with git rm when the project moved to Eclipse only
sync-to-github.bat	01.16-01.18	GitHub sync with private e-mail
CSC301_Group4_Monopoly_Data_Structures_Report.pdf	01.25, 03.02	Design document: data structures, recursion, Section 3 Twists, UML
CSC301_Group4_Monopoly_AI_Usage_Report_CS.pdf	01.23, 03.02	This report